Kingdom of Morocco

Sultan Moulay Slimane University

Multidisciplinary Faculty of Beni Mellal

Mini-Project Report

Reinforcement Learning for Dynamic Vehicle Routing under Uncertain Traffic Conditions

KHARBOUCH Essiddik, ER-RACHEDY Sana, BOUCHFIRA Hind, KHARBOUCH Yahya

MD3SI Master's Program — Module M122, Reinforcement Learning

Supervisors: Pr. Hamza ALLAGA, Pr. Meriem EL HARKAOUI

Abstract

Transportation and logistics account for a significant portion of the operational costs of distribution and delivery companies. Optimizing vehicle routes, known as the Vehicle Routing Problem (VRP), is a classic combinatorial optimization problem. However, traditional static approaches fail to adapt to real-time uncertainties related to stochastic demand and traffic. The objective of this work is to develop a routing agent capable of adapting its decisions to new traffic information while respecting a set of constraints: travel cost, delivery time, and vehicle capacity. We formalize the problem as a Partially Observable Markov Decision Process and use reinforcement learning to learn routing policies from interactions with a simulated environment. A Graph Neural Network (GNN) represents the relationships between the depot, customers, and routes. Two reinforcement learning methods, Double DQN and PPO, sharing this GNN encoder, were trained and compared against Random and Greedy Nearest-Neighbor baselines.

Over 100 held-out evaluation instances, GNN-DQN achieves a mean episode return of -1.69 (95% CI [-3.09, -0.30]), compared to -57.71 for Greedy Nearest-Neighbor and -72.27 for Random, a return improvement of 97% over the best baseline. This is driven primarily by time-window compliance: the late-delivery rate falls from 43.1% (Greedy) to 2.7% (GNN-DQN), while GNN-PPO reaches 12.5%. Notably, the learned agents do not achieve this by reducing travel cost, Greedy remains the cheapest by raw distance, but by learning to arrive early and wait rather than race between customers and risk missing a window, a behavior directly incentivized by the reward structure. GNN-DQN consistently outperforms GNN-PPO across training, though PPO was still improving at the training cutoff. These results show that under partial observability, distance-minimizing heuristics are insufficient for reliable time-window compliance, and that a learned policy can recover this compliance without an explicit model of traffic dynamics.

Keywords : *reinforcement learning, dynamic vehicle routing, partially observable MDP, graph neural networks, stochastic traffic*

Table of Contents

Figures list

Tables list

I. Introduction

The Dynamic Vehicle Routing Problem (DVRP) is an extension of the classical VRP in which customer requests, travel times, and traffic conditions are not fully known in advance. Information is revealed incrementally as routes are being executed, making it impossible to compute a single optimal solution upfront. The routing system must therefore operate as a reactive policy, continuously adjusting decisions as the environment evolves.

This project studies how Reinforcement Learning can be used to adapt vehicle routing decisions in real time to traffic that is both stochastic and only partially observable to the decision-maker, using a single capacitated vehicle serving a fixed set of customers from a depot as the running example.

The problem is addressed in two stages. First, the problem is formalized, a configurable simulation environment is built and validated, and two standard non-learning baselines are implemented and evaluated. Second, two deep-RL agents sharing a graph neural network encoder are trained and compared against these baselines and against each other.

II. Background and Related Work

1. MDP Formulation

MDP vs POMDP

We formalized our problem as a POMDP rather than a standard MDP, since the agent cannot observe the complete state of the environment at each step. The agent can observe the true traffic multiplier of an edge once that edge falls within the predefined observation radius of the vehicle's current position. Before entering this observation range, the traffic condition of the edge remains unknown to the agent and is represented by a masked value.

The other components of the state are fully observable, including the vehicle's current position, customer demands, remaining capacity, visited customers, elapsed time, and time-window constraints. Therefore, the agent has complete information about most aspects of the environment but only partial information about traffic conditions. This partial observability is the main reason for formulating the problem as a POMDP.

Real State vs Observation

The main difference between the state and the observation is the availability of traffic information. The environment maintains the complete traffic state, whereas the agent only observes traffic within its current revelation radius. Unknown traffic remains hidden until it is revealed through the vehicle's movement.

State : Customer positions, Customer demands, Customer time windows, ***Traffic conditions on all relevant edges***, Vehicle position, Remaining vehicle capacity, Visited customers, Current time.

$$S_t = (X, D, TW, \mathcal{T}_t, p_t, C_t, V_t, \tau_t)$$

Observation : Customer positions, Customer demands, Customer time windows, ***Observed traffic conditions***, Vehicle position, Remaining vehicle capacity, Visited customers, Current time.

$$O_t = \left(X, D, TW, \mathcal{T}_t^{\text{obs}}, p_t, C_t, V_t, \tau_t\right)$$

Action

The action space consists of a fixed-size discrete set containing all N nodes in the environment, including the depot and all customers. Its size remains unchanged throughout the episode, which allows the reinforcement learning models to use a fixed-size output layer. At each step, a separate action mask determines legal actions (served customers, over-capacity demands, and the current node are excluded; the depot is a legal target whenever the vehicle is not already there, to allow reloading).

Reward

The reward is a weighted sum of five terms, applied per step or per episode-end event

Table 1. Reward components and wights

Component	Sign	When it applies	weight
Travel cost (distance × traffic multiplier)	Negative, every step	Every move the vehicle makes.	w = 1.0
Time window violation	Negative	per occurrence.	w = 10.0
Successful delivery	Small positive bonus	Each time a customer is served.	w = 1.0
Unserved customer at episode end	Negative, per customer	Only if the episode is truncated (ran out of time) with people still unserved.	w = 20.0
Completing the full route	Positive bonus	Only if all customers served and vehicle back at depot.	w = 5.0

2. Algorithm Overview

Four routing methods are implemented and compared under identical conditions

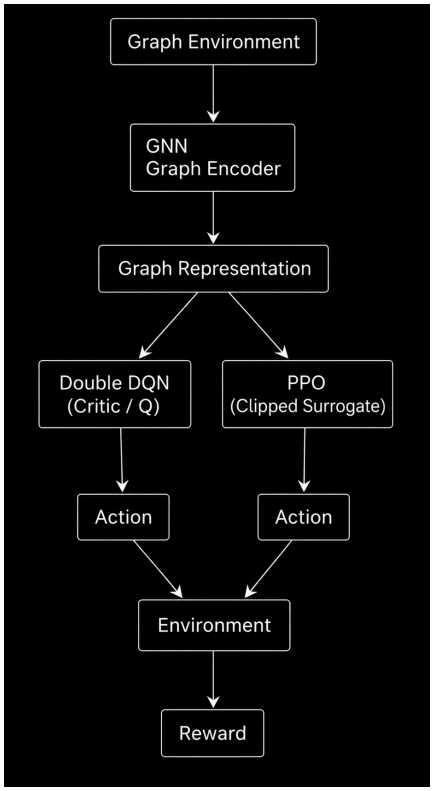
Table 2. Routing methods used in the experimental evaluation

Method	Type	Learning-based
Random Policy	Heuristic	No
Greedy Nearest-Neighbor	Heuristic	No
GNN-DQN	Value-based DRL (Double DQN)	Yes
GNN-PPO	Policy-based DRL (PPO)	Yes

DQN is a value-based method that learns Q(s,a) values for the possible routing actions. The agent selects the valid action with the highest Q-value. The implementation uses Double DQN, experience replay, and separate online and target networks.

PPO is an actor-critic method that learns a policy for selecting routing actions while a critic estimates the value of the current state. The policy is updated using PPO's clipped objective.

Figure 1 : Training Pipeline for RL-agents



Both agents use the same GNN-based state representation. The encoder uses node features and edge features describing distance and observed traffic, and produces node embeddings together with a global graph representation.

This setup allows the comparison to focus on the differences between value-based DQN and actor-critic PPO while keeping the environment and state representation consistent.

3. Related Work on the Proposed Modification

Existing reinforcement learning approaches to vehicle routing have demonstrated the ability of neural policies to learn effective routing decisions from randomly generated problem instances, while graph-based architectures have improved the representation of the relationships between customers and routes. However, most of these approaches assume relatively static or fully known travel conditions. Our proposed modification extends this setting to ***dynamic routing under uncertain and partially observed traffic***, using a GNN to represent the evolving road network and a per-node action-value (Double DQN) or policy (PPO) head conditioned on the cumulative observation, so that traffic revealed earlier in the episode continues to inform decisions later in the rout

III. Experimental Protocol

1. Environment Specification

The experiments use a Dynamic Capacitated Vehicle Routing Problem (DVRP) modeled as a partially observable decision-making problem(POMDP) under uncertain traffic. Each instance contains one depot and 15 customers on a 1.0 × 1.0 map. Customer demands range from 1 to 9 units, and the vehicle capacity is 30 units, with depot returns allowed for reloading.

Travel costs are based on Manhattan distance and a spatially correlated traffic field generated from [2–4] congestion centers. Traffic multipliers are normalized to [1.0, 3.0]. Traffic is partially observable: information is revealed only within a 0.3 observation radius around the vehicle's current position.

All customers have time windows, with starting times sampled from [0, 120] and widths from [30, 60]. Early arrivals require waiting, while late arrivals incur a penalty.

The action space contains one action per node, with an action mask excluding served customers, customers exceeding the remaining capacity, and the current node. The depot remains available for reloading. Observations include node features, distances, observed traffic, traffic masks, vehicle information, and the action mask.

An episode ends successfully when all customers are served and the vehicle returns to the depot. It is truncated after 200 actions, with an additional penalty for unserved customers.

The reward is primarily based on effective travel cost (distance × traffic multiplier). The main parameters are a travel-cost weight of 1.0, time-window penalty of 10.0, delivery bonus of 1.0, completion bonus of 5.0, and unserved-customer penalty of 20.0 per remaining customer.

2. Implementation

The codebase is organized into separate modules : src/env.py (environment), src/baselines.py (Random and Greedy Nearest-Neighbor policies behind a shared act(obs) interface), src/eval.py (evaluation loop and CSV logging).

The environment is implemented in Python using Gymnasium. NumPy is used for instance generation and numerical operations, while PyYAML is used to load the experimental configuration (full configuration).

The reinforcement-learning agents are implemented using PyTorch and PyTorch Geometric. Both GNN-DDQN and GNN-PPO use a Graph Neural Network (GNN) to process the routing state. DDQN estimates action values, while PPO learns a policy and a state-value function. The training procedures are implemented in src/train_dqn.py and src/train_ppo.py.

Training and evaluation use separate environment instances. Results are recorded in CSV format, including episode return, total routing cost, number of served customers, termination status, truncation status, and time-window outcomes. Model checkpoints are periodically saved for later evaluation.

3. Training Budget

The default training budget is 20,000 environment steps per run, with 1,000 warm-up steps before learning begins. The replay buffer stores up to 50,000 transitions, with a batch size of 64. The target network is updated every 500 steps, evaluation every 1,000 steps, and checkpoints every 5,000 steps.

4. Seeds

A fixed training seed of 42 is used for reproducibility across NumPy, PyTorch, and the replay buffer. Evaluation uses five deterministic seeds (42–46), with performance averaged across the resulting problem instances at each checkpoint.

5. Metrics

The experimental evaluation focuses on both routing efficiency and service quality. These metrics are directly produced by the environment and evaluation code and are stored in the experimental CSV results.

```
RESULTS_SCHEMA = [
    "method", "seed", "step", "episode", "episode_return","total_cost","n_served","n_customers",
    "terminated","truncated","n_on_time","n_waited","n_late" ]
```

Table 3. Performance metrics recorded during experimental evaluation

Metrics	Definition
Episode return	Cumulative reward obtained during an evaluation episode
Total routing cost	Accumulated effective travel cost, where each movement cost is the Manhattan distance multiplied by the corresponding traffic multiplier
Number of served customers	Number of customers successfully visited and served during the episode
Completion rate	Proportion of evaluation episodes that terminate successfully after serving all customers and returning to the depot
Truncation rate	Proportion of episodes reaching the maximum step limit without successful completion
On-time deliveries	Number of customers served without violating their time window
Waiting deliveries	Number of deliveries for which the vehicle arrives before the time window and therefore has to wait
Late deliveries	Number of customers served after the end of their time window

6. Hyperparameters

Table 4. GNN-DQN Frozen Hyperparameters

Hyperparameter	Value
Learning rate	0.001
Discount factor (γ)	0.99
Batch size	64
Replay buffer size	50,000
Warm-up steps	1,000
Total training steps	20,000
Target network update	500 steps
Epsilon (ϵ) [initial-final]	1.00-0,05
Epsilon decay steps	10,000
GNN layers	2
GNN hidden dimension	32
Edge feature dimension	3
Edge-network hidden dimension	32
Q-head hidden dimension	32
Training seed	42

Table 5. GNN-POO Frozen Hyperparameters

Hyperparameter	Value
Learning rate	0.003
Discount factor (γ)	0.99
GAE λ	0.95
Rollout length	512
Batch size	64
Update epochs	10
Clip coefficient	0.20
Value-loss coefficient	0.50
Entropy coefficient	0.01
Max gradient norm	0.50
Total training steps	20,000
Evaluation frequency	500 steps
Checkpoint frequency	5,000
Training seed	42

IV. Baseline Reproduction

Random Policy selects uniformly among currently legal actions (no learning, no memory) and serves as a lower-bound reference. Greedy Nearest-Neighbor always moves to the legal node with the smallest raw Manhattan distance from the vehicle's current position, matching the cahier des charges' literal definition ("closest unserved customer"), i.e. distance-based, not traffic-cost-adjusted. Both were implemented behind a common act(observation) interface so that the same evaluation loop will later run the learned agents without modification.

Table 6. Baseline performance comparison (over 100 seeds)

Metric	Random (n=100)	Greedy NN (n=100)	Δ (Greedy vs Random)
Episode return, mean ± std	-72.27 ± 22.50	-57.71 ± 21.41	+20.1%
Episode return, 95% CI	[-76.68, -67.87]	[-61.91, -53.52]	
Total travel cost, mean ± std	22.77 ± 5.12	13.01 ± 2.57	-42.9%
Late-delivery rate	46.3%	43.1%	-3.2 pp
Demand fulfillment / completion	100/100 (100%)	100/100 (100%)	0 pp

V. Proposed Enhancement

Motivation: the baselines reproduction section shows that a purely distance-based heuristics (Greedy Nearest-Neighbor) reduces travel cost substantially relative to Random Policy, but leaves the late-delivery rate largely unaddressed (43.1% vs. 46.3%). Neither baseline has any mechanism for reasoning about which customers are becoming time-critical; both select actions using only the current position and remaining demand. The proposed enhancement targets this specific gap.

GNN-DQN and GNN-PPO share a graph neural network encoder composed of two edge-conditioned NNConv layers. Each layer uses the edge attributes [distance,observed traffic,is observed], allowing messages between nodes to depend on the distance and current traffic state of the corresponding edge. The two layers are computed as:

```
x = LayerNorm₁( ReLU( layer1(node_feats, edge_index, edge_attr) ) )
x = LayerNorm₂( ReLU( layer2(x, edge_index, edge_attr) ) )
```

The current node's embedding, a mean-pooled representation of the whole graph, and the vehicle state remaining capacity and elapsed time are concatenated into a context vector. Each node's own embedding is then

combined with this context and passed to a per-node decision head. The head outputs a Q-value for each candidate node in GNN-DQN, or action logits and a state-value estimate in GNN-PPO.

Both agents use action masking to exclude illegal actions, including already served customers, demands exceeding the remaining vehicle capacity, and the current node. GNN-DQN uses Double DQN with a periodically updated target network, whereas GNN-PPO uses a clipped policy objective with Generalized Advantage Estimation.

Hypothesis. We hypothesized that the GNN encoder, by using progressively observed traffic information, would help the learned agents prioritize time-critical deliveries and reduce lateness relative to the heuristic baselines. This hypothesis is examined in Section VII.

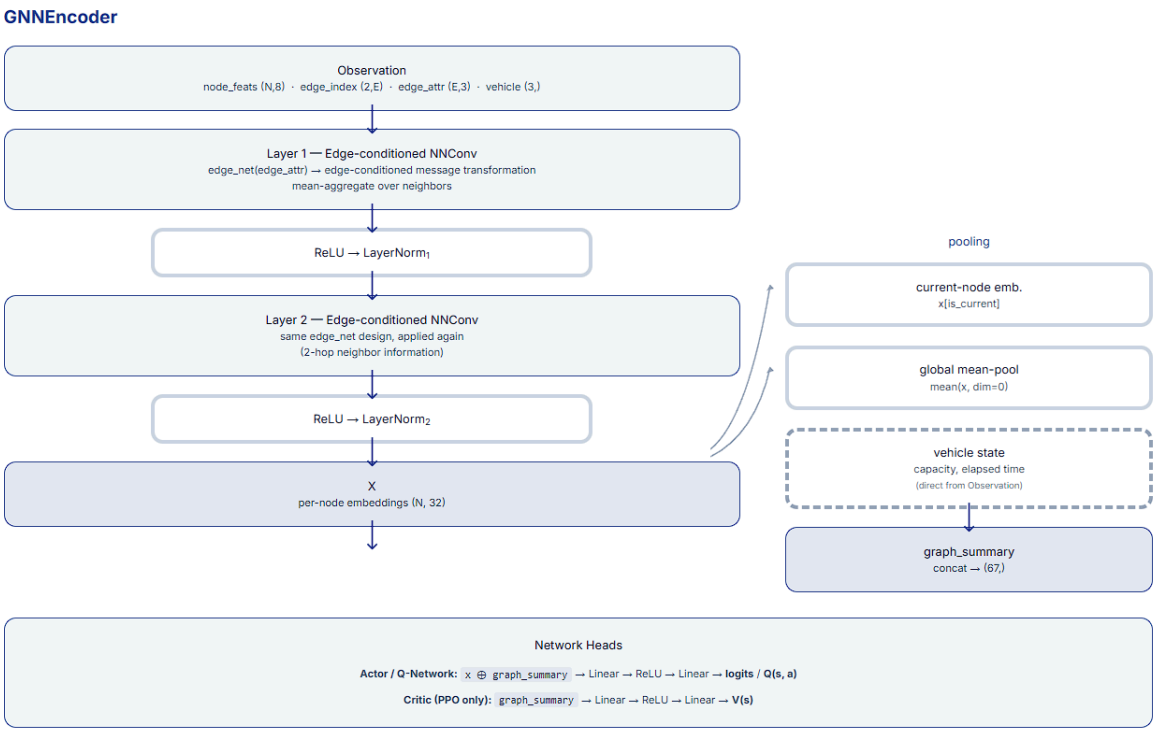


Figure2 : Architecture of the GNN-based Encoder and Decision Heads

This figure shows the implemented GNN encoder. Two NNConv layers turn the graph observation into an embedding for each node. The current-node embedding, the average graph embedding, and the vehicle state are combined into ***graph_summary***. This context is used with each node embedding to select the next node in GNN-DQN and GNN-PPO; PPO also uses it to estimate the state value.

VI. Results

Baseline results are reported in full in Section IV. This section presents the comparative evaluation of all four methods (Random Policy, Greedy Nearest-Neighbor, GNN-DQN, and GNN-PPO), on the same 100 held-out evaluation seeds (42-141), together with the training dynamics of the two learned agents.

1. Evaluation Results

Table 7. Comparative evaluation of baselines and learned routing methods (over 100 seeds)

Method	Return (mean ± std)	Cost (mean ± std)	On-Time	Waited	Late	Late_rate
Random	-72.27 ± 22.50	22.77 ± 5.12	5.2	2.8	7.0	46.3%
Greedy NN	-57.71 ± 21.41	13.01 ± 2.57	5.4	3.1	6.5	43.1%
GNN-PPO	-19.91 ± 18.02	21.21 ± 4.84	7.9	5.2	1.9	12.5%
GNN-DQN	-1.69 ± 7.13	17.59 ± 3.36	7.3	7.3	0.4	2.7%

Both learned agents outperform the two heuristic baselines on *episode return* and *late-delivery rate*. **GNN-DQN** improves mean return by approximately **97%** relative to Greedy Nearest-Neighbor (the stronger baseline) and reduces the late-delivery rate from 43.1% to 2.7%. GNN-PPO shows the same qualitative improvement but at a smaller magnitude a 65.5% gain in return and a late rate of 12.5%.

Neither learned agent achieves this improvement through lower travel cost : GNN-DQN's cost (17.59) is 35.2% higher than Greedy's, and GNN-PPO's (21.21) is 63.0% higher. The on-time/wait/late breakdown suggests why.

2. Training Dynamics

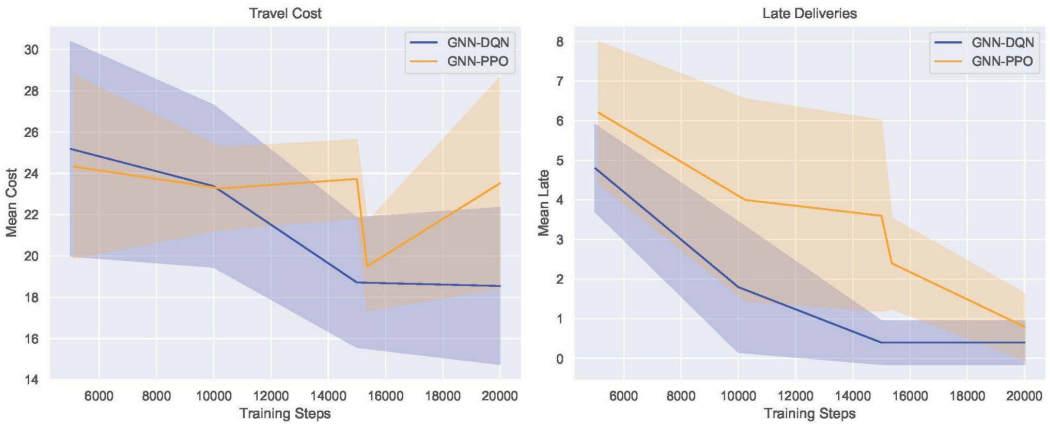


Figure3 : Learning curve metrics (GNN-DQN & GNN-PPO)

This figure shows how the two learned agents evolve during training in terms of *travel cost* and the number of *late deliveries.* GNN-DQN shows a relatively stable reduction in both metrics, Its mean travel cost decreases from approximately 25 at 5,000 training steps to about 18.5 at 20,000 steps. At the same time, the mean number of late deliveries falls from 4.8 to 0.4. The reduction becomes particularly clear after 10,000 training steps, and the curves become more stable around 15,000 steps.

GNN-PPO also improves during training, but its progress is less stable. Its travel cost fluctuates between checkpoints and remains higher than that of GNN-DQN at the end of training, at approximately 23.5. However, its late-delivery count decreases from 6.2 at the first checkpoint to 0.8 at 20,000 steps. The larger shaded region for GNN-PPO indicates greater variability across the five evaluation seeds. Overall, both methods learn to reduce late deliveries, but GNN-DQN reaches a lower cost and fewer late deliveries within the available training budget.

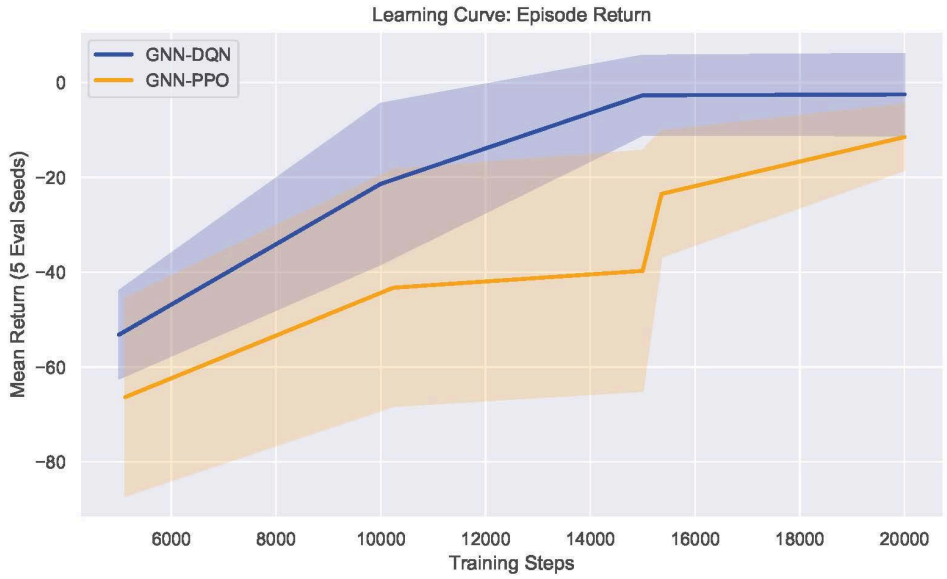


Figure4 : Learning curve return (GNN-DQN & GNN-PPO)

Presenting the mean episode return during training. Both agents improve over successive training steps; however, DQN improves more rapidly and reaches a higher final return than PPO.

GNN-DQN's mean return increases from approximately −53.2 at 5,000 training steps to −2.6 at 20,000 steps. Most of this improvement occurs before 15,000 steps, after which the curve remains close to zero, suggesting convergence. In comparison, GNN-PPO improves from −66.3 at 5,120 steps to −11.5 at 20,000 steps. Although PPO remains below GNN-DQN at the end, the shaded regions represent the standard deviation across five evaluation seeds at each checkpoint.

VII. Ablation Study

To determine whether the performance gains reported in *Results Section* are attributable to the agent's use of traffic-observability information, rather than to other components of the state representation, an ablation was conducted in which the trained GNN-DQN checkpoint was re-evaluated, without retraining, under five values of ***observation_radius,*** including complete blindness (0.0), on the same 100 held-out seeds used throughout this report.

Table 8. Sensitivity of the trained GNN-DQN policy to the traffic-observation radius

Radius	Return (mean ± std)	Cost (mean ± std)	Late_rate
0.00 (fully blind)	-1.93 ± 7.42	17.73 ± 3.55	2.8%
0.15	-1.77 ± 7.30	17.57 ± 3.57	2.8%
0.30 (trained)	-1.69 ± 7.16	17.59 ± 3.37	2.7%
0.50	-3.02 ± 8.32	18.02 ± 3.65	3.3%
1.50 (near-full observability)	-11.63 ± 10.20	24.43 ± 5.07	4.8%

Conclusion: Performance at radius 0.00, 0.15, and 0.30 is essentially the same, the differences between them are far smaller than the natural run-to-run variation(the expected variation in average performance across 100 random episodes, even for an unchanged policy), so they are not meaningful. In other words, taking away traffic information completely does not make the agent perform any worse. Performance only gets worse once the radius goes above the value used in training slightly at 0.50, and clearly at 1.50, where cost increases by 38.9% and the late-delivery rate rises by 2.1% points.

This changes how the earlier results should be read. If the agent's good performance came from actually using real-time traffic information, then removing that information should have hurt it, but it did not, even when the agent could see no traffic at all. This suggests that GNN-DQN's advantage over the baselines (Section VI) comes probably from reasoning about time windows, deciding when to wait, based on each customer's deadline, the current time, and known distance none of which require seeing traffic. This is an honest limitation worth stating clearly: the traffic-aware part of the architecture contributes far less to the final result than it was designed to.

VIII. Discussion

The results indicate that the learned agents improve service quality mainly by managing delivery timing rather than by minimizing raw travel cost. Greedy Nearest-Neighbor achieves the lowest travel cost, but its distance-based decisions do not sufficiently account for time-window constraints. In contrast, GNN-DQN and GNN-PPO obtain better returns and substantially fewer late deliveries, although they accept higher travel costs. Their larger number of waiting deliveries suggests that they often arrive early and wait, which is preferable to arriving late under the current reward function.

GNN-DQN performs better than GNN-PPO within the available 20,000-step training budget. Its learning curve becomes relatively stable after approximately 15,000 steps, whereas PPO continues to improve near the end of training. Therefore, the observed difference should not be interpreted as evidence that Double DQN is universally superior to PPO. It may partly reflect the different convergence speeds of the two algorithms and the limited training budget.

The ablation study provides an important qualification. Changing the traffic-observation radius from 0.00 to 0.30 produces almost identical results, while larger radii lead to worse performance. This means that the final GNN-DQN policy does not appear to rely strongly on the observed traffic features. Its improvement over the baselines is more likely related to its ability to represent customer states, deadlines, vehicle time, and waiting decisions. Consequently, the experiment supports the effectiveness of the learned routing policy, but it does not provide strong evidence that traffic observability itself caused the improvement.

Threats to Validity

- **Seeds:** Results use 100 held-out evaluation seeds and one training seed per agent. Training curves show variation across five evaluation seeds.
- **Environment scope:** Experiments use one synthetic environment with one vehicle and 15 customers. Results may not generalize to larger, multi-vehicle, or real-world routing problems.
- **Hyperparameters:** The chosen settings were tuned for this environment and may not work equally well under different conditions.
- **Training budget:** PPO was still improving at the final checkpoint, so longer training could reduce its gap with DQN.

Overall, the experiments show that the learned agents are more effective than the heuristic baselines at satisfying time-window constraints in the tested environment. However, the evidence suggests that this gain comes primarily from learned timing and waiting decisions, rather than from the use of traffic-observability information itself. The traffic-aware mechanism therefore remains a promising but not yet fully validated component of the proposed approach.

IX. Conclusion and Future Work

The project developed and evaluated GNN-based reinforcement learning agents for dynamic vehicle routing problems with uncertain traffic and delivery time windows. GNN-DQN and GNN-PPO were compared with Random Policy and Greedy Nearest-Neighbor under the same held-out evaluation seeds. Both learned agents improved episode return and strongly reduced late deliveries compared with the heuristic baselines. GNN-DQN obtained the best final performance, with a mean return of −1.69 and a late-delivery rate of 2.7%, compared with −57.71 and 43.1% for Greedy Nearest-Neighbor.

The results show that minimizing distance alone is not sufficient when deliveries must respect time windows. The learned agents accepted higher travel costs but reduced lateness by arriving early and waiting when necessary. However, the observation-radius ablation showed that removing traffic information did not strongly reduce DQN performance. Therefore, the current results demonstrate that our proposed model approach learns effective timing and waiting decisions, which greatly reduce late deliveries. In this environment, the results suggest that time windows, vehicle state, and learned routing decisions are more important than traffic visibility.

Future work should train each agent with longer training budgets, especially for PPO. Further ablation studies should examine the contribution of time-window features, waiting behaviour, reward weights, vehicle capacity, traffic intensity, and the GNN encoder. The environment could also be extended to larger routing instances, multiple vehicles, real road networks, and real traffic data.

References

[1] Nazari, M., Oroojlooy, A., Snyder, L., & Takáč, M. (2018). Reinforcement Learning for Solving the Vehicle Routing Problem.

[2] Solomon, M. M. (1987). Algorithms for the Vehicle Routing and Scheduling Problems with Time Window Constraints.

[3] Wouda, N. A., Lan, L., & Kool, W. (2024). PyVRP: A High-Performance VRP Solver Package. INFORMS Journal on Computing

[4] PyVRP Documentation, Introduction to VRP.

[5] Sultan Moulay Slimane University, FP Béni Mellal (2025). Mini-Project Requirements: Reinforcement Learning for Dynamic Vehicle Routing under Uncertain Traffic Conditions.

Appendix

1. Full Hyperparameter Configuration (YAML)

```
seed: 42 # fixed seed

map:

    size: 1.0

problem:
    n_customers: 15
    vehicle_capacity: 30
    demand_low: 1
    demand_high: 9

traffic:
    low: 1.0
    high: 3.0
    spatial_scale: 0.30
    observation_radius: 0.3

time_windows:
    enabled: true
    window_width_low: 30
    window_width_high: 60
    earliest_start: 0
    latest_start: 120

episode:

    max_steps: 200

evaluation:

    n_episodes: 100

reward:
    w_distance: 1.0
    w_time_window_penalty: 10.0
    w_delivery_bonus: 1.0
    w_complete_bonus: 5.0
    w_unserved_penalty: 20.0

Training:           # Training hyperparameters (GNN-DQN)
    total_steps: 20000
    warmup_steps: 1000
    buffer_size: 50000
    batch_size: 64
```

```
    learning_rate: 0.001
    gamma: 0.99
    epsilon_start: 1.0
    epsilon_end: 0.05
    epsilon_decay_steps: 10000
    target_update_freq: 500
    eval_every: 1000
    checkpoint_every: 5000
    seed: 42

PPO:                    # Training hyperparameters (GNN-PPO)
    total_steps: 20000
    rollout_length: 512
    batch_size: 64
    n_epochs: 10
    learning_rate: 0.0003
    gamma: 0.99
    gae_lambda: 0.95
    clip_eps: 0.2
    vf_coef: 0.5
    ent_coef: 0.01
    max_grad_norm: 0.5
    eval_every: 500
    checkpoint_every: 5000
    seed: 42
```

2. Per-Seed Raw Final Returns

First 8 of 100 evaluated seeds per method (full data: results/baselines.csv, 200 rows):

Table 9. Per-Seed Raw Final Returns

Method	Seed	episode_return	total_cost
Random	42	-41.71	31.71
Random	43	-45.18	15.18
Random	44	-59.42	29.42
Random	45	-50.47	20.47
GreedyNN	42	-13.88	13.88
GreedyNN	43	-32.38	12.38
GreedyNN	44	-42.78	12.78
GreedyNN	45	-39.45	9.45

3. Reproduction Command

```
# 1. Setup
pip install -r requirements.txt
# 2. Train GNN-DQN (writes checkpoints/dqn/, logs/dqn/)
python -m src.train_dqn --config configs/default.yaml
# 3. Train GNN-PPO (writes checkpoints/ppo/, logs/ppo/)
python -m src.train_ppo --config configs/default.yaml
# 4. Evaluate both agents + baselines
python -m src.eval      # writes results/baselines.csv
# Alternative: run the full pipeline (train + eval) end-to-end
./run_all.bat      #windows powershell
bash run_all.sh      #Linux terminal
# Note: same config (configs/default.yaml, seed 42) reproduces identical
# per-episode instances and results, verified across repeated runs.
```

4. Compute Budget

Baselines: negligible (< 1 minute, CPU only, no training). GNN-DQN / GNN-PPO training budget

Table 10. Compute Budget

Platform	Local windows machine
GPU type	APU : AMD Ryzen 5 4650U
Total training time	~ 3hours ● GNN-DQN : ~ 2hours 30min ● GNN-PPO : ~ 20min
Number of episodes/timesteps trained	40 000 total timesteps ● 20 000 for DQN and 20 000 for PPO

5. Contribution Statement

Task	Sana Er-rachedy	Essiddik Kharbouch	Hind Bouchfira	Yahya Kharbouch
Environment & POMDP design (env.py)	yes	yes	no	no
Baselines (baselines.py)	no	no	yes	yes
Evaluation infrastructure (eval.py, eval_all_checkpoints.py)	yes	yes	no	no
GNN encoder (gnn_encoder.py)	yes	yes	no	no
GNN-DQN implementation & training	yes	yes	no	no
GNN-PPO implementation & training	no	no	yes	yes
Ablation study (observation-radius sweep)	yes	yes	no	no
Figures & visualization (plot.py, visualize.py)	no	no	yes	yes
Reproducibility (run_all.sh/.bat, requirements.txt, configs)	no	yes	no	no
Report writing	yes	yes	yes	yes
Lab notebook (notebook.md)	yes	yes	yes	yes

All members contributed to the cahier des charges review, weekly lab-notebook entries, and final report review. LLM assistance was used throughout to support code editing, language correction, report structure, and review of clarity.